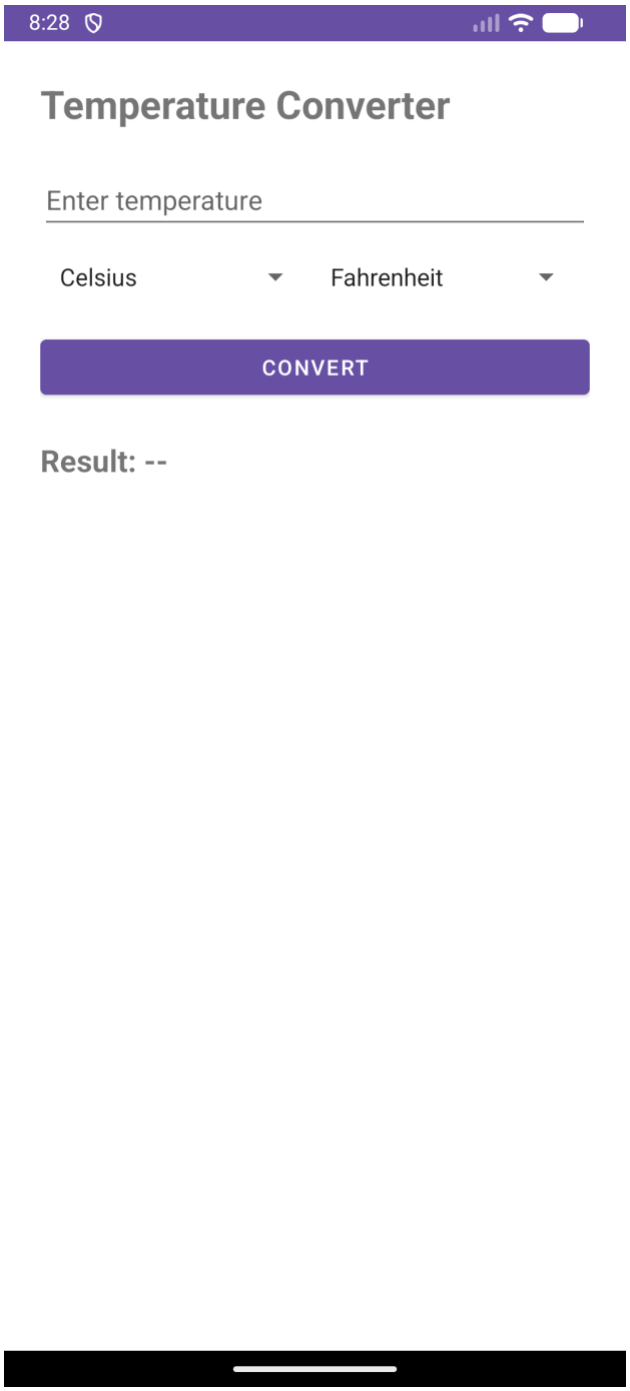
COMP1786 Logbook Excercise 1
Basic Information

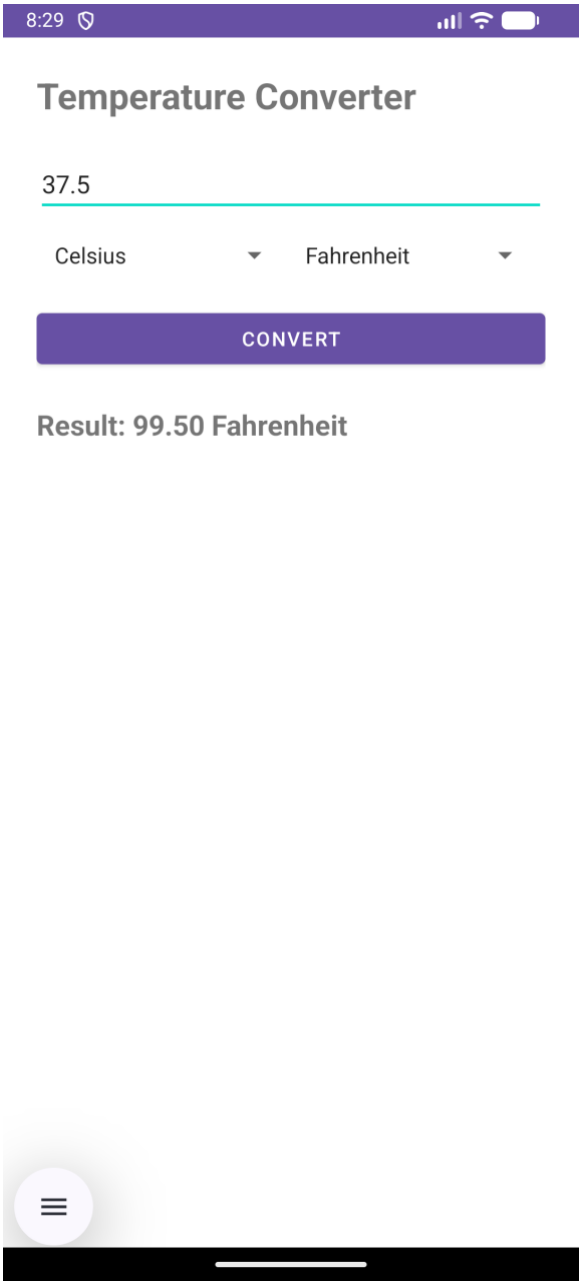
1.1 Student name	**Le Anh Khoa**
1.2 Who did you work with? Note that for logbook exercises you are allowed to work with one other person as long as you give their name and login id and both contribute to the work.	**Name:** **Login id:**
1.3 Which Exercise is this?	• Exercise Exercise 1 - Temperature converter application
1.4 How well did you complete the exercise? Tick as appropriate.	• I tried but couldn't complete it [ ] • I did it but I feel I should have do better [ ] • I did everything that was asked [x] • I did more than was asked for [ ]
1.5 Briefly explain your answer to question 1.4 Without any explanation/justification, your scores will be deducted.	**I have successfully developed the Temperature Converter application using Android Studio and Java. The application features a user-friendly interface that allows users to input a numerical temperature value and convert it between Celsius, Fahrenheit, and Kelvin using Spinner dropdown menus. Furthermore, I implemented robust input validation to prevent system crashes; if the user leaves the input field blank, the app displays a clear error message (using setError and Toast) to guide the user appropriately.**

2. Exercise answer

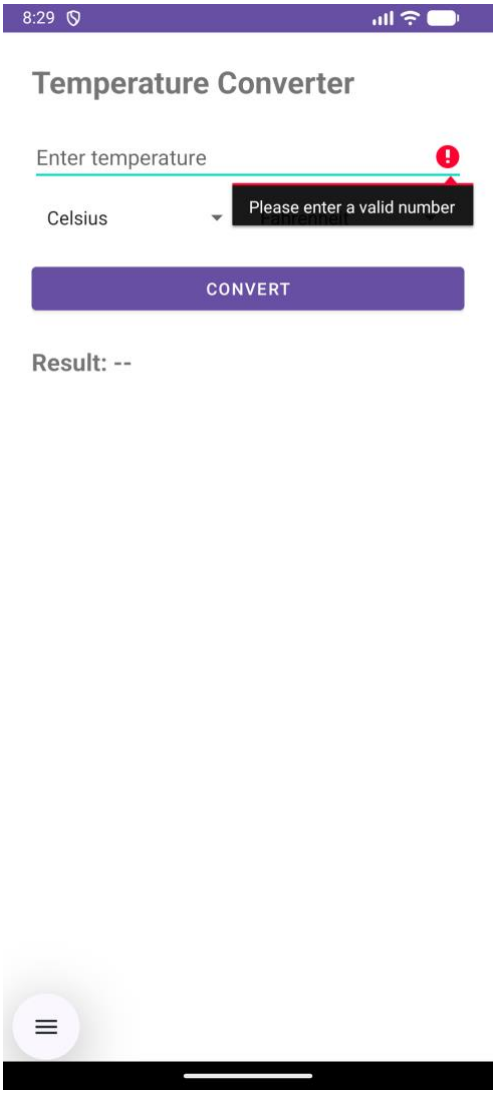
2.1 Screen shots demonstrating what you achieved


8:28
Temperature Converter
Enter temperature
Celsius
Fahrenheit
CONVERT
Result: --

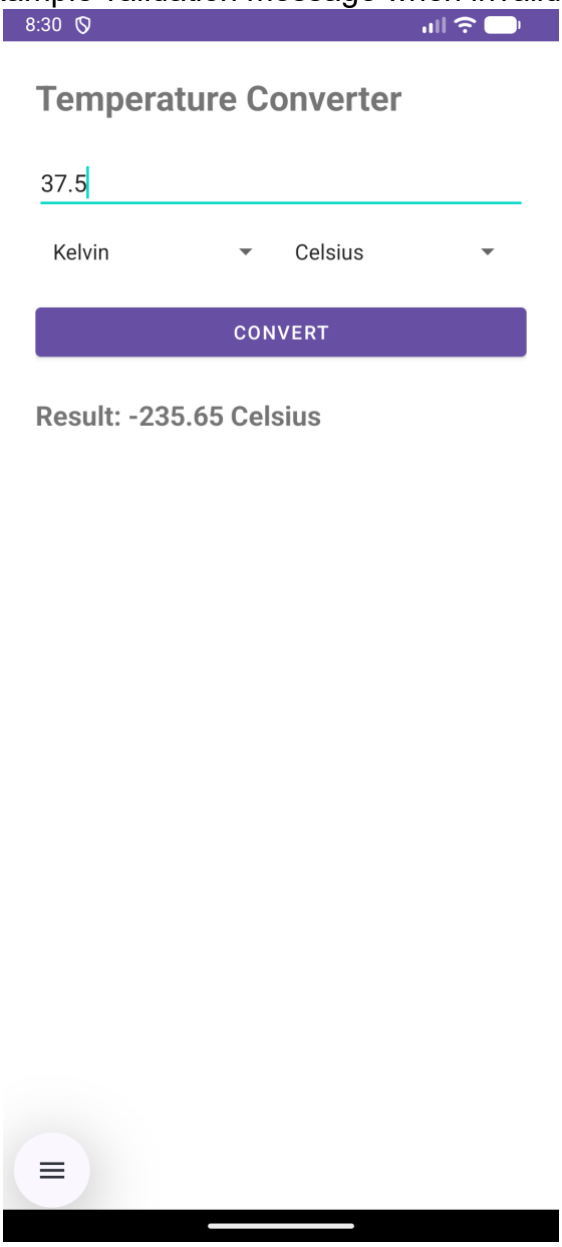
Screenshot 1: Main temperature converter screen showing input field, scale selectors and convert button.



Screenshot 2: Example conversion from Celsius to Fahrenheit.


8:29
Temperature Converter
Enter temperature
Please enter a valid number
Celsius
CONVERT
Result: --

Screenshot 3: Example validation message when invalid input is entered.



Screenshot 4: Example conversion from Kelvin to Celsius.

2.2 Code that you wrote

File: MainActivity.java
This file contains the main conversion logic. It reads the user input, validates it, converts between the selected temperature scales, and displays the result.

```
private double convertTemperature(double value, String fromUnit, String toUnit) {

    double celsius;


    if (fromUnit.equals("Celsius")) {
```

```
        celsius = value;
    } else if (fromUnit.equals("Fahrenheit")) {
        celsius = (value - 32) * 5 / 9;
    } else {
        celsius = value - 273.15;
    }

    if (toUnit.equals("Celsius")) {
        return celsius;
    } else if (toUnit.equals("Fahrenheit")) {
        return celsius * 9 / 5 + 32;
    } else {
        return celsius + 273.15;
    }
}
```

File: MainActivity.java

This validation code checks whether the user entered a valid number before conversion. It helps prevent crashes and improves usability.

```
private double convertTemperature(double value, String fromUnit, String toUnit) {
    double celsius;

    if (fromUnit.equals("Celsius")) {
        celsius = value;
    } else if (fromUnit.equals("Fahrenheit")) {
        celsius = (value - 32) * 5 / 9;
    } else {
        celsius = value - 273.15;
    }

    if (toUnit.equals("Celsius")) {
        return celsius;
    } else if (toUnit.equals("Fahrenheit")) {
        return celsius * 9 / 5 + 32;
    } else {
        return celsius + 273.15;
    }
}
```

Exercise 1 – Explanation

This exercise demonstrates a simple but complete Android application that converts temperature values between Celsius, Fahrenheit and Kelvin. I used appropriate input controls and validation so that the user can enter a value safely and select units clearly, which improves usability and reduces common input

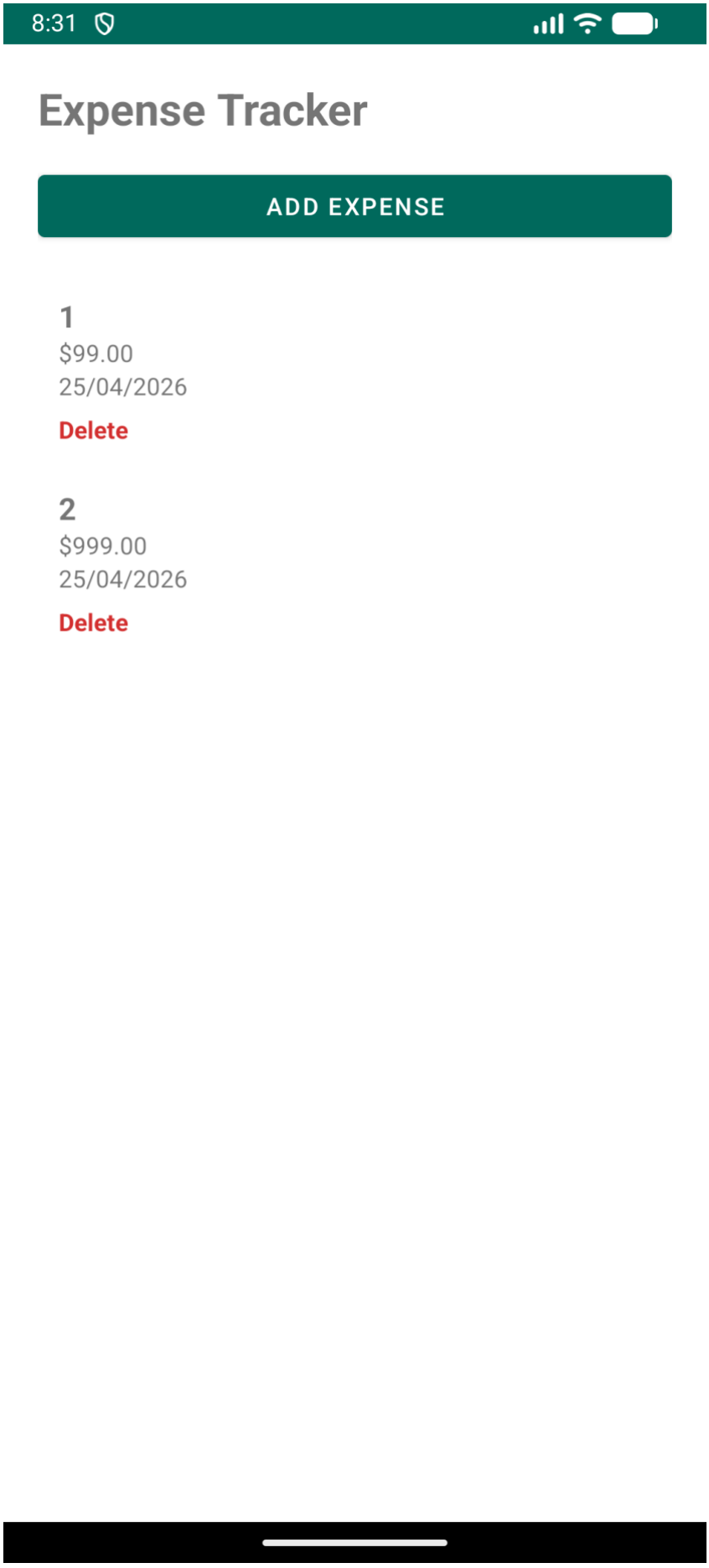
mistakes. The implementation also shows understanding of event-driven Android development because the conversion is triggered by user interaction and the result is updated immediately on screen (Android Developers, 2026a; Material Design, 2026a). Overall, this exercise helped me understand how activities manage interface behaviour and how simple logic can be connected to a practical mobile user interface (Android Developers, 2026b).

COMP1786 Logbook Excercise 2
Basic Information

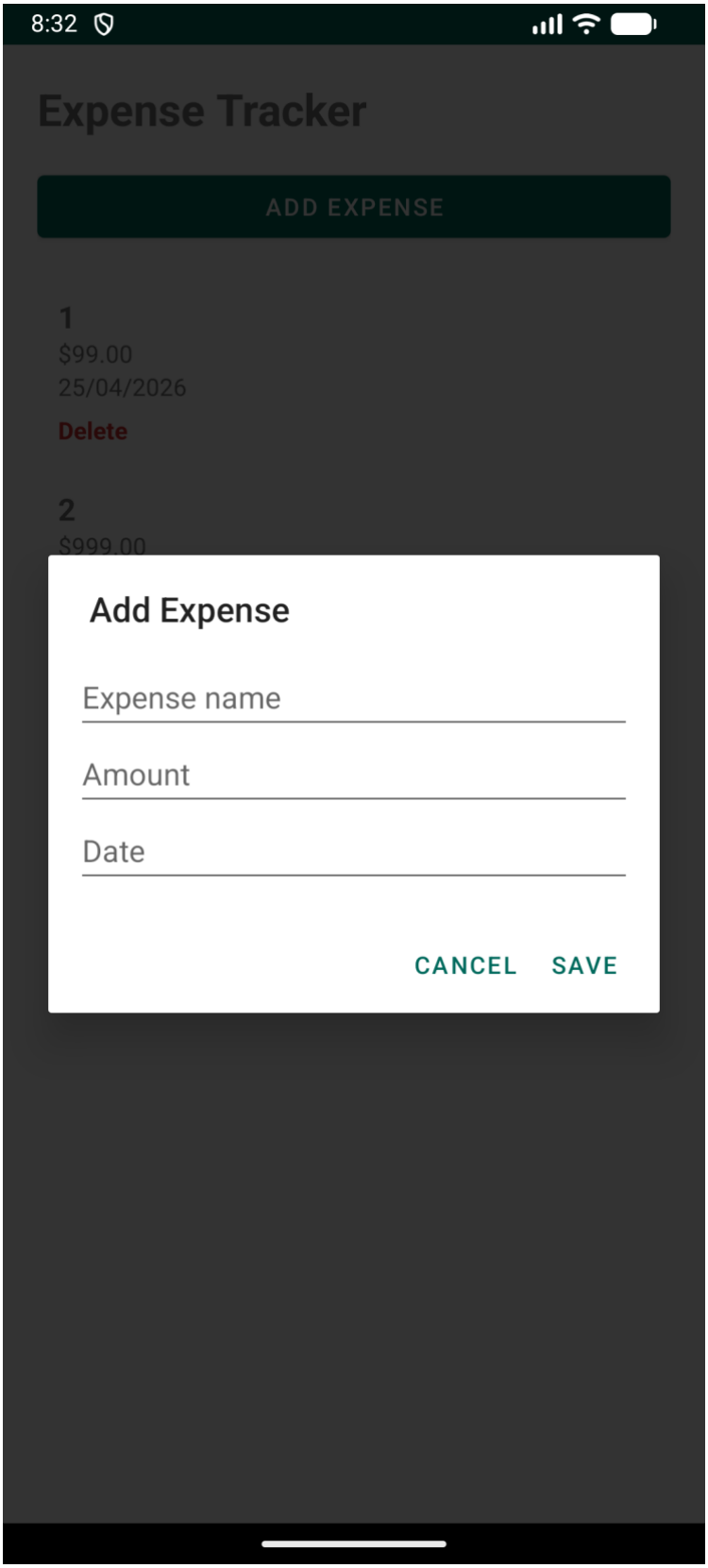
1.1 Student name	**Le Anh Khoa**		
1.2 Who did you work with? Note that for logbook exercises you are allowed to work with one other person as long as you give their name and login id and both contribute to the work.	**Name: None** **Login id: None**		
1.3 Which Exercise is this?	• Exercise 2 – Simple Expense Tracker Application		
1.4 How well did you complete the exercise? Tick as appropriate.	• I tried but couldn't complete it • I did it but I feel I should have do better • I did everything that was asked • I did more than was asked for	 X 	
1.5 Briefly explain your answer to question 1.4 Without any explanation/justification, your scores will be deducted.	**I extended the previous expense tracker by adding persistent storage using SQLite. I added a category field for each expense and made sure data is saved locally and reloaded when the application opens again. Users can add, update and delete entries, and all changes are reflected in the database. In addition to the basic requirements, I also improved validation and used a more structured data layer for database operations.**		

3. Exercise answer

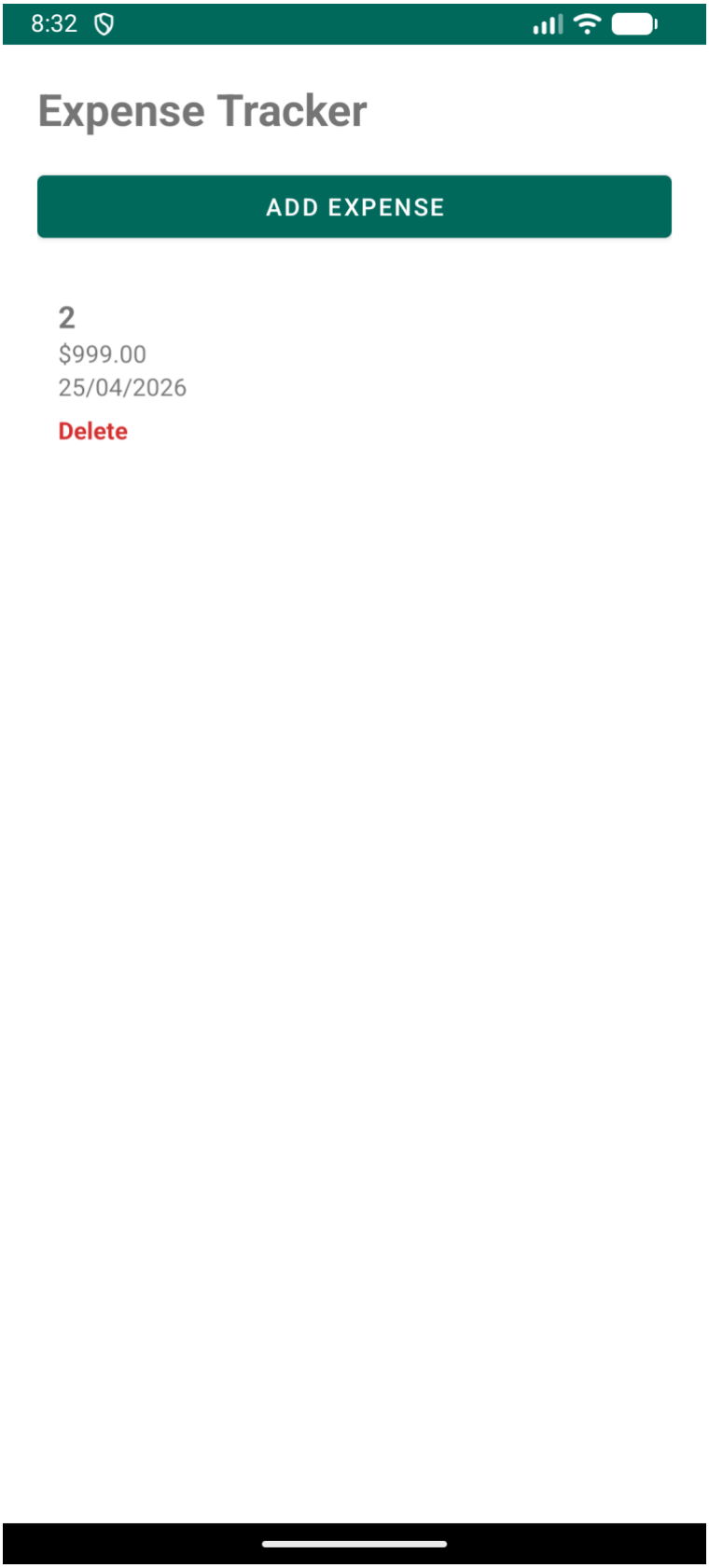
2.1 Screen shots demonstrating what you achieved


8:31
Expense Tracker
ADD EXPENSE
1
$99.00
25/04/2026
Delete
2
$999.00
25/04/2026
Delete

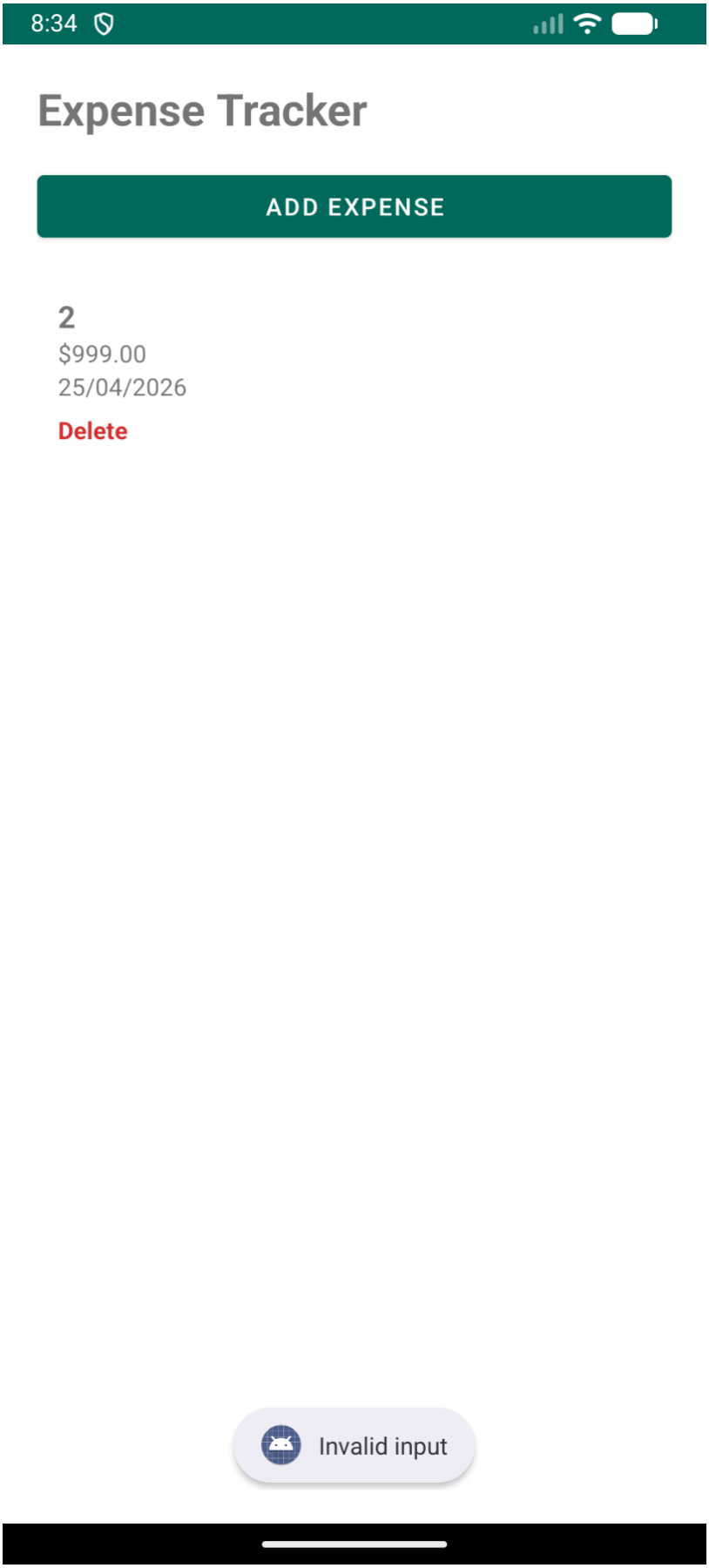
Screenshot 1: Main expense tracker screen showing the list of expenses.



Screenshot 2: Add expense dialog/form with fields for name, amount and date.



Screenshot 3: Example of deleting an expense entry.



Screenshot 4 : Validation message when amount or date is invalid.

3.2 Code that you wrote

File: Expense.java
This class represents a single expense entry. It stores the name, amount and date of each item.

```
public class Expense {

    private String name;

    private double amount;

    private String date;


    public Expense(String name, double amount, String date) {

        this.name = name;

        this.amount = amount;

        this.date = date;

    }


    public String getName() { return name; }

    public double getAmount() { return amount; }

    public String getDate() { return date; }


    public void setName(String name) { this.name = name; }

    public void setAmount(double amount) { this.amount = amount; }

    public void setDate(String date) { this.date = date; }

}
```

File: MainActivity.java
This code handles adding a new expense to the list after validating the input fields.

```
private void addExpense() {

    String name = etName.getText().toString().trim();

    String amountText = etAmount.getText().toString().trim();

    String date = etDate.getText().toString().trim();


    if (name.isEmpty()) {

        etName.setError("Expense name is required");

        return;
```

```
    }

    if (amountText.isEmpty()) {
        etAmount.setError("Amount is required");
        return;
    }

    try {
        double amount = Double.parseDouble(amountText);
        expenseList.add(new Expense(name, amount, date));
        adapter.notifyDataSetChanged();
    } catch (NumberFormatException e) {
        etAmount.setError("Invalid amount");
    }
}
```

File: ExpenseAdapter.java

This adapter displays the expense list in a RecyclerView. It shows the name, amount and date for each expense item.

```
@Override

public void onBindViewHolder(@NonNull ExpenseViewHolder holder, int position) {

    Expense expense = expenseList.get(position);

    holder.tvName.setText(expense.getName());

    holder.tvAmount.setText(String.valueOf(expense.getAmount()));

    holder.tvDate.setText(expense.getDate());

}
```

Exercise 2 – Explanation

This exercise extends my understanding from a single-function app to a data-based application that manages multiple expense entries. I implemented add, edit and delete operations so that the user can interact with a dynamic list, which is a common mobile pattern for financial or personal-record applications. The use of a structured expense model together with a list adapter shows an understanding of object-oriented design and efficient list presentation in Android applications (Oracle, 2026a; Android Developers, 2026c; Material Design, 2026b). In addition, dialog-based input and field validation make the

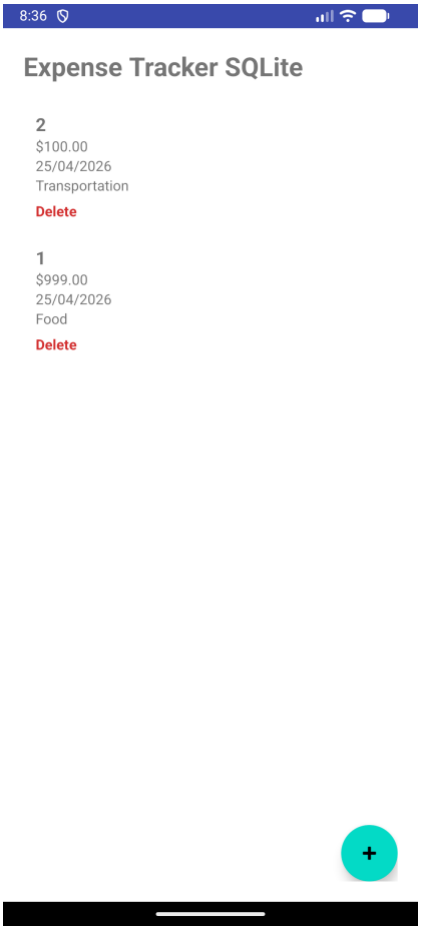
application more practical and consistent with recommended Android interaction design (Android Developers, 2026d).

COMP1786 Logbook Excercise 3
Basic Information

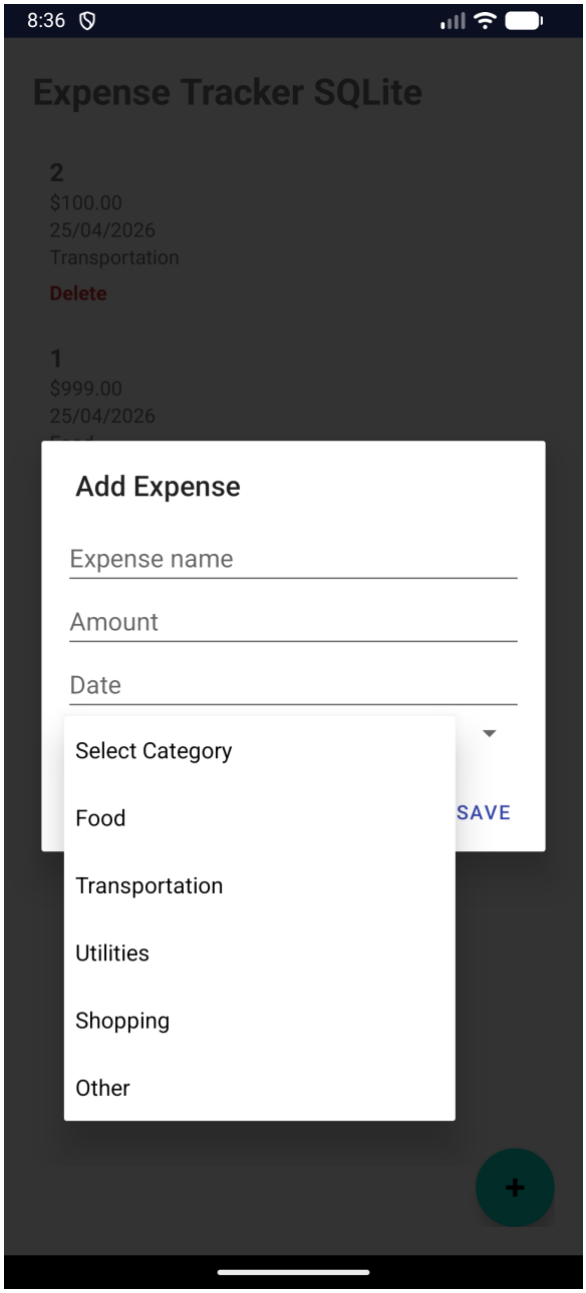
1.1 Student name	**Le Anh Khoa**
1.2 Who did you work with? Note that for logbook exercises you are allowed to work with one other person as long as you give their name and login id and both contribute to the work.	**Name: None** **Login id: None**
1.3 Which Exercise is this?	• Exercise Exercise 1 - Temperature converter application
1.4 How well did you complete the exercise? Tick as appropriate.	• I tried but couldn't complete it [ ] • I did it but I feel I should have do better [ ] • I did everything that was asked [ ] • I did more than was asked for [x]
1.5 Briefly explain your answer to question 1.4 Without any explanation/justification, your scores will be deducted.	**I extended the previous expense tracker by adding persistent storage using SQLite. I added a category field for each expense and made sure data is saved locally and reloaded when the application opens again. Users can add, update and delete entries, and all changes are reflected in the database. In addition to the basic requirements, I also improved validation and used a more structured data layer for database operations.**

4. Exercise answer

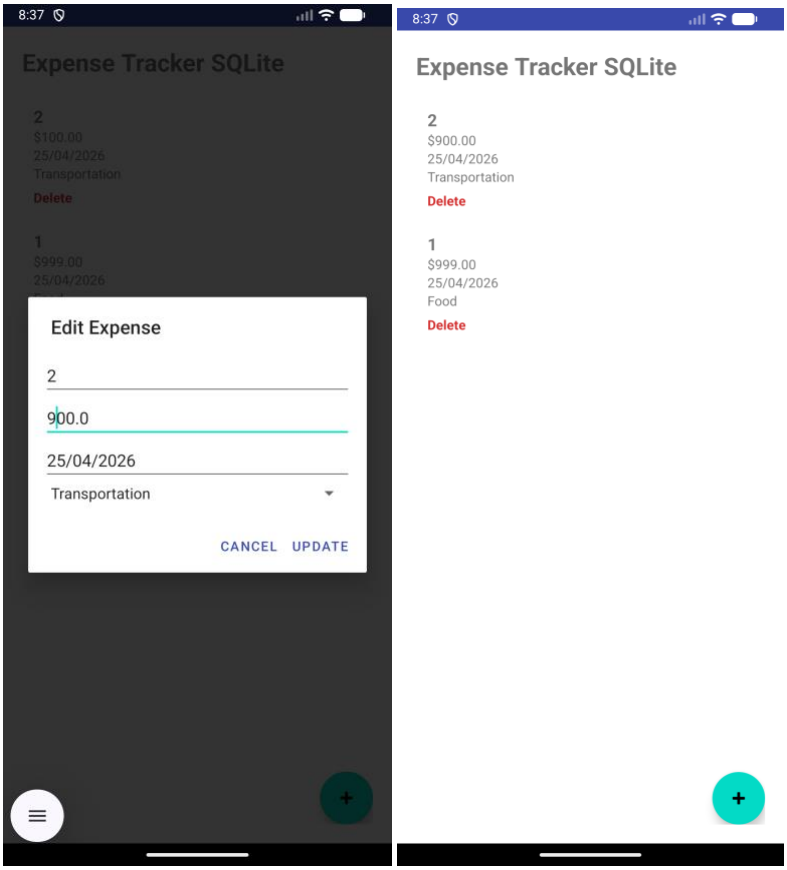
2.1 Screen shots demonstrating what you achieved



Screenshot 1: Main screen showing saved expenses loaded from SQLite.



Screenshot 2: Add expense form including category selection.



Screenshot 3: Example of updating an expense entry and showing updated data in the list.

Screenshot 4: Example of deleting an entry from the database.

Screenshot 5: Validation example for invalid amount, missing category or incorrect date.

4.2 Code that you wrote

file: DatabaseHelper.java
This file creates the SQLite database and the expenses table. It is important because it defines the structure used to persist the data locally.

```
public class DatabaseHelper extends SQLiteOpenHelper {

    private static final String DB_NAME = "expenses.db";

    private static final int DB_VERSION = 1;



    public static final String TABLE_EXPENSES = "expenses";

    public static final String COL_ID = "id";

    public static final String COL_NAME = "name";

    public static final String COL_AMOUNT = "amount";

    public static final String COL_DATE = "date";

    public static final String COL_CATEGORY = "category";


    public DatabaseHelper(Context context) {

        super(context, DB_NAME, null, DB_VERSION);

    }


    @Override

    public void onCreate(SQLiteDatabase db) {

        String sql = "CREATE TABLE " + TABLE_EXPENSES + " (" +

                COL_ID + " INTEGER PRIMARY KEY AUTOINCREMENT, " +

                COL_NAME + " TEXT, " +

                COL_AMOUNT + " REAL, " +

                COL_DATE + " TEXT, " +

                COL_CATEGORY + " TEXT)";

        db.execSQL(sql);
```

```
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS " + TABLE_EXPENSES);
        onCreate(db);
    }
}
```

File: ExpenseRepository.java

This code inserts expense data into SQLite. It is used whenever the user saves a new entry.

```
public boolean insertExpense(Expense expense) {
    SQLiteDatabase db = dbHelper.getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put(DatabaseHelper.COL_NAME, expense.getName());
    values.put(DatabaseHelper.COL_AMOUNT, expense.getAmount());
    values.put(DatabaseHelper.COL_DATE, expense.getDate());
    values.put(DatabaseHelper.COL_CATEGORY, expense.getCategory());

    long result = db.insert(DatabaseHelper.TABLE_EXPENSES, null, values);
    return result != -1;
}
```

File: ExpenseRepository.java

This method retrieves all saved expense entries from the database when the app opens again.

```
public ArrayList<Expense> getAllExpenses() {
    ArrayList<Expense> list = new ArrayList<>();
    SQLiteDatabase db = dbHelper.getReadableDatabase();
    Cursor cursor = db.query(DatabaseHelper.TABLE_EXPENSES, null, null, null, null, null, null);

    if (cursor.moveToFirst()) {
        do {
            String name = cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.COL_NAME));
            double amount =
cursor.getDouble(cursor.getColumnIndexOrThrow(DatabaseHelper.COL_AMOUNT));
            String date = cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.COL_DATE));
            String category =
cursor.getString(cursor.getColumnIndexOrThrow(DatabaseHelper.COL_CATEGORY));
            list.add(new Expense(name, amount, date, category));
```

```
        } while (cursor.moveToNext());
    }

    cursor.close();
    return list;
}
```

File: MainActivity.java
This code validates the user input before saving the expense to SQLite.

```
private boolean validateExpenseInput(String name, String amount, String date, String category) {
    if (name.isEmpty()) {
        etName.setError("Name is required");
        return false;
    }
    if (amount.isEmpty()) {
        etAmount.setError("Amount is required");
        return false;
    }
    if (date.isEmpty()) {
        etDate.setError("Date is required");
        return false;
    }
    if (category.equals("Select Category")) {
        Toast.makeText(this, "Please select a category", Toast.LENGTH_SHORT).show();
        return false;
    }
    return true;
}
```

Exercise 3 – Explanation

This exercise improves the previous expense tracker by adding SQLite persistence, meaning the expense data remains available after the application is closed and reopened. I used a database helper and repository-style data access so that storage logic is separated more clearly from user-interface logic, which makes the app easier to maintain and extend. This demonstrates understanding of local mobile data management, table structure design, and CRUD operations using SQLite inside Android applications (Android Developers, 2026e; Android Developers, 2026f; SQLite, 2026). The final version is therefore more realistic than Exercise 2 because it supports long-term data storage instead of temporary in-memory data

only (Oracle, 2026b; Android Developers, 2026g).

References

Android Developers (2026a) Introduction to activities. Available at: https://developer.android.com/guide/components/activities/intro-activities

Android Developers (2026b) Activity. Available at: https://developer.android.com/reference/android/app/Activity

Android Developers (2026c) Create dynamic lists with RecyclerView. Available at: https://developer.android.com/develop/ui/views/layout/recyclerview

Android Developers (2026d) Dialogs. Available at: https://developer.android.com/develop/ui/views/components/dialogs

Android Developers (2026e) Data and file storage overview. Available at: https://developer.android.com/training/data-storage

Android Developers (2026f) SQLiteOpenHelper. Available at: https://developer.android.com/reference/android/database/sqlite/SQLiteOpenHelper

Android Developers (2026g) Save data using SQLite. Available at: https://developer.android.com/training/data-storage/sqlite

Material Design (2026a) Text fields. Available at: https://m3.material.io/components/text-fields/overview

Material Design (2026b) Lists. Available at: https://m3.material.io/components/lists/overview

Oracle (2026a) What Is a Class? Available at: https://docs.oracle.com/javase/tutorial/java/concepts/class.html

Oracle (2026b) Exceptions. Available at: https://docs.oracle.com/javase/tutorial/essential/exceptions/